Projective Geometry as a Unified Framework for Exact Computational Geometry

From incidence to Cayley-Klein metrics and rational trigonometry

Wai-Shing Luk

projgeom (Python / Rust / C++)

October 11, 2026

Section 1

Outline

Outline

1. Outline
2. Motivation ✡
3. The Projective Plane
4. Classical Theorems
5. Conics and Cayley-Klein
6. Rational Trigonometry
7. Software Architecture
8. Discussion

Section 2

Motivation



Geometry Is Fragmented — and It Need Not Be

- **Cayley:** *"Projective geometry is all geometry."*
- Euclidean, elliptic and hyperbolic geometry are one projective space with one quadratic form and three metrics
- Rational trigonometry measures **without** square roots or trigonometric functions [1]
- The whole computational core reduces to three operations: **dot**, **cross**, **parametrize**
- Don't shoot rabbits with machine guns: pick the measurement that fits the task 🎯

Contributions

1. Coordinate-first projective plane: incidence, duality, join and meet from dot/cross
2. Pappus, Desargues, Pascal and quadrangular sets as polynomial identities
3. Conics $\rightarrow$ polarity $\rightarrow$ the **Cayley-Klein** family of metric geometries
4. Rational trigonometry: the five laws, Archimedes, Heron, and Ptolemy
5. A **four-layer** software architecture (Python $\cdot$ Rust $\cdot$ C++) with exact arithmetic
6. Cross-language verification (30/30), property-based tests, and microbenchmarks

Section 3

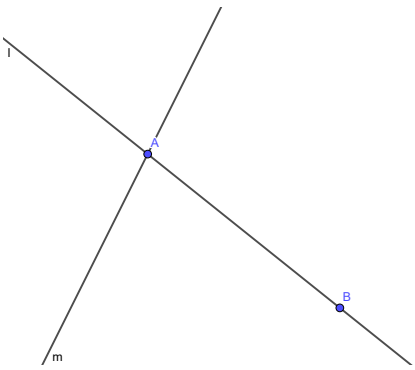
The Projective Plane

Homogeneous Coordinates

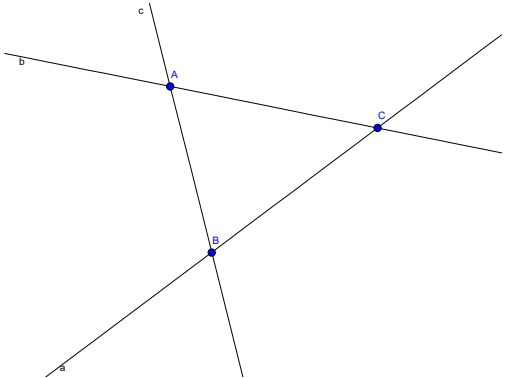
- A point $p = [x, y, z]$ and a line $l = [a, b, c]$ are nonzero 3-vectors over a field K
- $[x, y, z] = \lambda[x, y, z]$ for all $\lambda \neq 0$; the triple $[0, 0, 0]$ is forbidden
- **Incidence:** p lies on l iff $p \cdot l = ax + by + cz = 0$
- Affine plane: $[x, y, 1]$; direction $[x, y, 0]$ is a *point at infinity*; $l_\infty = [0, 0, 1]$
- Euclidean geometry is just one normalization, one choice of the line at infinity

Join, Meet, and Duality

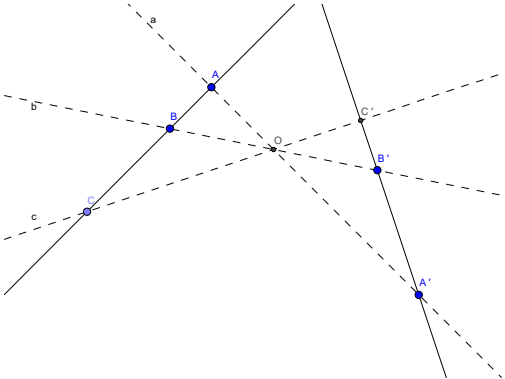
- Join of points: $[p, q] = p \times q$ — a line
- Meet of lines: $[l, m] = l \times m$ — a point
- **It is the same operation**; duality swaps point ↔ line for free
- $p \cdot (p \times q) = 0$: the join is incident with both
- $p = q \iff p \times q = 0$: equality by cross product



Triangles, Duality and Perspectivity



A triangle and its dual trilateral.



A perspectivity from a center.

Points on a Line; Any Field

- Parametrization $\lambda p + \mu q$ spans the line $[p, q]$ — the workhorse of every construction
- Holds over $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, and finite fields $\mathrm{GF}(p)$
- No multiplicative inverse is needed for incidence, join or meet — a **ring** suffices
- Integer coordinates give exact arithmetic with no rounding

Section 4

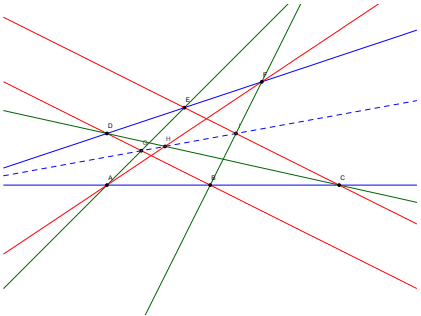
Classical Theorems

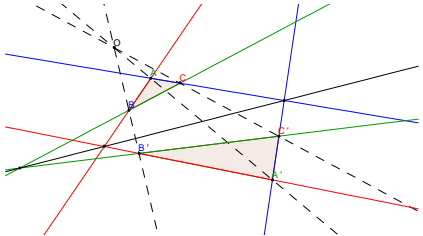
Pappus and Desargues

Pappus. If A, B, C are collinear and D, E, F are collinear, then $AE \cap BD$, $AF \cap CD$, $BF \cap CE$ are collinear.

Desargues. Two triangles are perspective from a point iff they are perspective from a line. Self-dual.

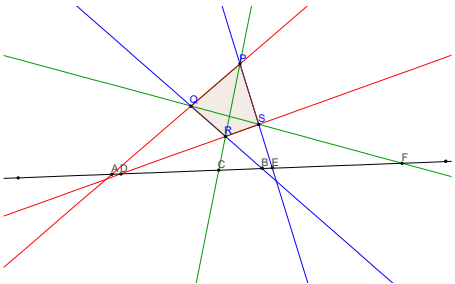
- Proof: parametrize and expand; $X \cdot (Y \times Z)$ simplifies to 0 — an **identity over any field**





Quadrangular Sets and Cross Ratio

- A complete quadrangle meets a line in a **quadrilateral set**
- Collapse two pairs: a **harmonic set** $H(AB, CF)$
- Cross ratio
$$R(A, B; C, D) = \frac{(OA \cdot C)(OB \cdot D)}{(OA \cdot D)(OB \cdot C)}$$
- Equals -1 exactly for a harmonic range
- Harmonic conjugates are computed from two dot products

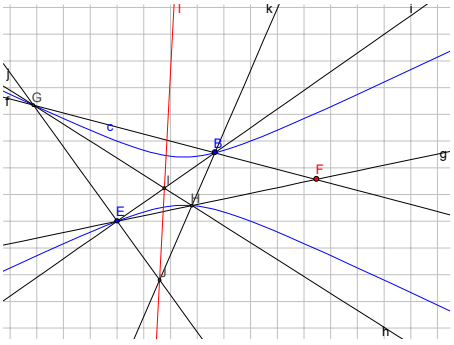


Section 5

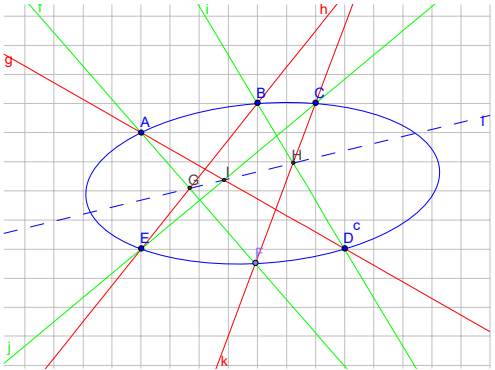
Conics and Cayley-Klein

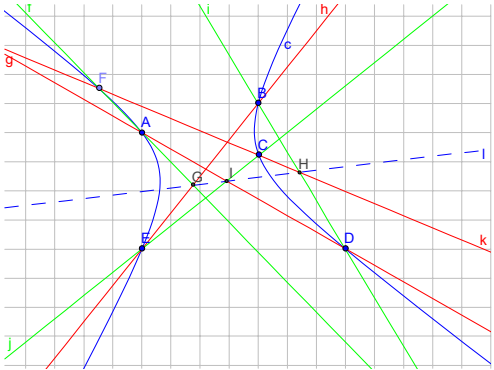
Conics and Polarity

- A **polarity** is a correlation of period two: $a = A^{\perp}$, $A = a^{\perp}$
- Self-conjugate points form a **conic**; the polars of conic points are its tangents
- Construct polar, pole and tangent with secants — no metric required



Pascal's Theorem





- A hexagon inscribed in a conic: the opposite sides meet in collinear points
- The conic counterpart of Pappus; it degenerates to Pappus when the conic splits into two lines

The Fundamental Cone

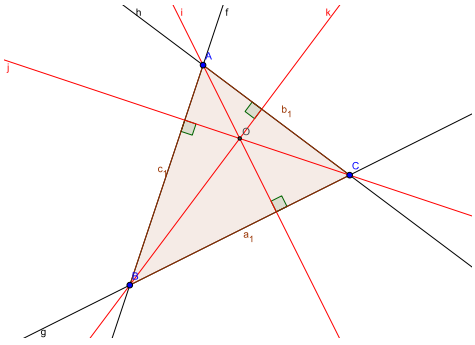
- Cayley-Klein geometry = a projective plane plus a fundamental conic $(\mathbf{A}, \mathbf{B})$
- $[A^\perp] = \mathbf{A}[A]$ and $[a^\perp] = \mathbf{B}[a]$ — one matrix-vector product

Geometry	Fundamental conic	$p^\perp$
Elliptic	$x^2 + y^2 + z^2 = 0$	(x, y, z)
Hyperbolic	$x^2 + y^2 - z^2 = 0$	$(x, y, -z)$
Euclidean	$z = 0$ (degenerate)	$[0, 0, 1]$

- Only the action of $\perp$ differs; the incidence structure is common

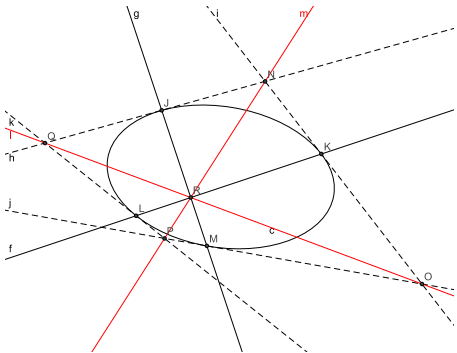
Perpendicularity, Altitude, Orthocenter

- $l \perp m \iff l^{\perp}$ is incident with m
- Altitude of l through p = join $[p, l^{\perp}]$
- The three altitudes of a triangle are concurrent: the **orthocenter**
- The relation is involutive on the triangle
- Works in elliptic and hyperbolic geometry too

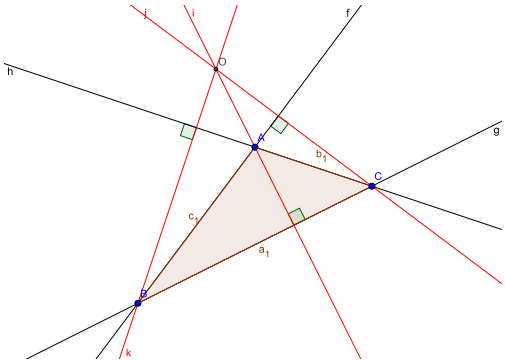


Involution, Reflection, Midpoint, Bisector

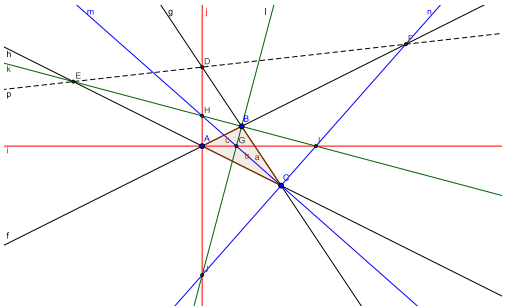
- Involution: $\tau(\tau(p)) = p$, with matrix $T = (o^{\mathsf{T}} m)\,\mathrm{I} - 2\,o\,m^{\mathsf{T}}$
- Reflection = involution centered at $m^{\perp}$ (Euclidean: a point at infinity)
- Midpoints: $r = \sqrt{\Phi(p)}\,q \pm \sqrt{\Phi(q)}\,p$, where $\Phi(p) = p^{\mathsf{T}}\mathbf{A}p$
- Bisectors are the dual construction



Two Theorems in the Cayley-Klein Plane



The orthocenter relation is involutive on the triangle.



The complete angle-bisector theorem.

Section 6

Rational Trigonometry

Quadrance and Spread

- $\Omega(x) = x \cdot x^{\perp}$ measures the fundamental conic
- Quadrance: $q(A, B) = \dfrac{\Omega(AB)}{\Omega(A)\,\Omega(B)}$ — squared distance
- Spread: $s(l, m) = \dfrac{\Omega(lm)}{\Omega(l)\,\Omega(m)}$ — squared sine of angle
- Euclidean $q = d^2$; hyperbolic $q = \sinh^2 d$; elliptic $q = \sin^2 d$
- **No square root, no trigonometric function** — rational in the coordinates

The Five Laws

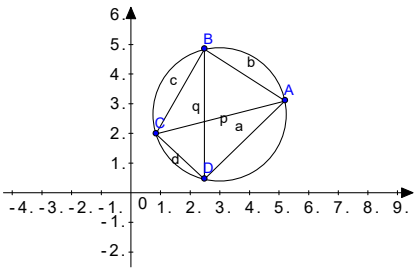
- Pythagoras: $q_3 = q_1 + q_2 - q_1 q_2$
- Triple quad: $(q_1 + q_2 + q_3)^2 = 2(q_1^2 + q_2^2 + q_3^2) + 4q_1 q_2 q_3$
- Triple spread: $(s_1 + s_2 + s_3)^2 = 2(s_1^2 + s_2^2 + s_3^2) + 4s_1 s_2 s_3$
- Spread law: $q_1/s_1 = q_2/s_2 = q_3/s_3$
- Cross law: $(s_1 s_2 q_3 - (s_1 + s_2 + s_3) + 2)^2 = 4(1 - s_1)(1 - s_2)(1 - s_3)$
- In Euclidean geometry the last term of the triple quad formula drops out

Archimedes, Heron, Ptolemy

- $\mathrm{Ar}(q_1, q_2, q_3) = 4q_1q_2 - (q_1 + q_2 - q_3)^2$
- $16 \cdot \text{area}^2 = \mathrm{Ar}(q_1, q_2, q_3)$; Heron's formula generalizes it
- **Ptolemy (rational):** a quadrilateral is cyclic iff

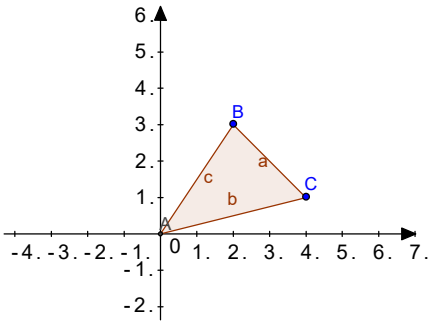
$$\mathrm{Ar}(Q_{12}Q_{34},\, Q_{23}Q_{14},\, Q_{13}Q_{24}) = 0$$

- The three arguments enter **symmetrically** — convexity is no longer needed



Heron's Formula

- The area of a triangle from its side lengths: $16 \cdot \text{area}^2 = \text{Ar}(Q_1, Q_2, Q_3)$
- Brahmagupta generalizes it to a cyclic quadrilateral
- Both are specializations of the Archimedes function

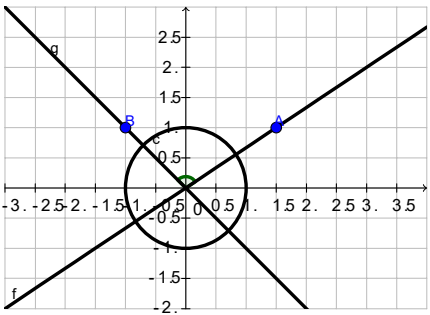


The Projective Line and Relativity

- $\mathbb{P}^1$: cross ratio R_1 and the Mobius transformation $z' = \dfrac{az+b}{cz+d}$
- Velocity is $[x,t]$ with $v = x/t$; the relative speed is

$$\text{Speed}(p,q) = \frac{(v_p - v_q)^2}{(v_p^2-1)(v_q^2-1)}$$

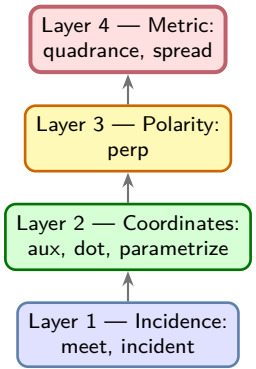
- With $|v| \leq 1$ the relative speed never exceeds one
- Additivity is replaced by the projective composition of Mobius maps



Section 7

Software Architecture

Four Layers = Four Axiom Levels



- Each layer adds **exactly** the operations its axiom level needs, and nothing more
- A pure projective object cannot compute a perpendicular — the type system rejects it, so the compiler becomes a geometry proof assistant

One Line of `perp()` Defines a Geometry

Object	Coordinates	`perp()`
Elliptic point	(x, y, z)	same coordinates
Hyperbolic point	(x, y, z)	$(x, y, -z)$
Euclidean point	(x, y, z)	$[0, 0, 1]$

- Everything above `perp()` — altitude, orthocenter, reflection, measure — is **shared**
- Three lines of definition give elliptic, hyperbolic and Euclidean geometry
- Rust macro `impl_cayley_klein_plane!` implements all four layers at once

Cross-Language Verification and Benchmarks

- **30 / 30** shared reference cases agree across Python, Rust and C++
- **163** collected tests, **91** of them Hypothesis property tests of algebraic invariants
- Microbenchmarks:

Operation	C++ (ns)	Python (μs)	Ratio
Dot product	2.6	1.16	440
Cross product	2.5	1.71	680
Parametrize	2.5	2.79	1,110
Harmonic conjugate	2.9	13.2	4,650

- C++ and Rust: a few nanoseconds; Python: below 150 microseconds

The Cost of Exactness

- Rational arithmetic = gcd normalization + overflow guards
- The division-based guard fired **0 times in 2,000,000** operations — pure overhead
- Neutralizing it recovers about 20% on small values, 10% on large ones
- It also hid a **division-by-zero crash**: $(1/2) + 0$ faulted
- Replacing it with a multiply-overflow intrinsic (`idivq` → `imulq` + `seto`) fixes both
- Removing a redundant gcd: compound operations get 27–57% faster
- **Profile first, then remove the right work** 🔬

Section 8

Discussion

What the Framework Does Not Do

- **Exactness stops at the rationals:** no sqrt, trig, float or symbolic layer. Rational polar/pole, conic-line intersection (square discriminant) and the 4-point projective transform are exact; irrational or complex results are refused, not approximated
- **Confined to $\mathbb{P}^2$:** $\mathbb{P}^3$ needs Grassmann-Cayley / Plucker coordinates; "join = meet = cross" is a two-dimensional accident
- **Coordinate growth:** each join/meet doubles the degree; bit-length explodes without interpolation or filtered predicates
- **Non-additive measurements:** quadrance and spread are quadratic; distance and angle are recovered only at the boundary

Takeaways 🎯

- One representation, one pair of operations, one hierarchy
- Metric geometries differ **only** in `perp()` — the fundamental conic
- Measurement is rational by default; exactness is a deliberate **scope choice**
- The architecture makes illegal geometry unrepresentable
- Same algorithms, three languages: [2], [3], [4]

References

Key background: Coxeter [5], Richter-Gebert [6], Yap [7].

[1] N. J. Wildberger, *Divine proportions: Rational trigonometry to universal geometry*. Sydney: Wild Egg Books, 2005.

[2] W.-S. Luk, "Projgeom-py: Projective geometry in python." https://github.com/luk036/projgeom-py, 2026.

[3] W.-S. Luk, "Projgeom-rs: Projective geometry in rust." https://github.com/luk036/projgeom-rs, 2026.

[4] W.-S. Luk, "Projgeom-cpp: Projective geometry in c++." https://github.com/luk036/projgeom-cpp, 2026.

[5] H. S. M. Coxeter, *Projective geometry*, 2nd ed. New York: Springer-Verlag, 1974.

[6] J. Richter-Gebert, *Perspectives on projective geometry: A guided tour through real and complex geometry*. Berlin, Heidelberg: Springer, 2011.

[7] C.-K. Yap, *Fundamental problems of algorithmic algebra*. New York: Oxford University Press, 2000.

Q&A 🙋

- Repositories: `github.com/luk036/projgeom-{py,rs,cpp}`
- Slides + paper: `github.com/luk036/projgeom-py/tree/main/paper`

Thank you! 🎉